convolutional blocks, followed by a *maxpooling* layer and an activation function called *relu*. The input size considered for an image is 224 x 224:

```
new = Sequential()
new.add(Conv2D(32, (3, 3), input_shape=(224, 224,3)))
new.add(BatchNormalization())
new.add(MaxPooling2D())

new.add(Conv2D(32, (3, 3), activation='relu'))
new.add(MaxPooling2D())

new.add(Conv2D(32, (3, 3), activation='relu'))
new.add(MaxPooling2D())

new.add(Conv2D(64, (3, 3), activation='relu'))
new.add(MaxPooling2D())
```

As mentioned earlier in the article, the above part of the CNN mode performs feature learning and generates a feature map. After feature learning, it is required to classify images using fully connected layers. For that, we have added two dense layers. The classification problem mentioned here is a type of binary classification; therefore, we have used sigmoid as an activation function. To convert the feature map into the fully connected layer, we have to first flatten the array of the former. After that, using a dense layer, we add two fully connected layers:

```
new.add(Flatten())
new.add(Dense(64))
new.add(Activation('relu'))
new.add(Dense(32))
new.add(Activation('relu'))

new.add(Dense(1))
new.add(Activation('sigmoid'))
new.summary();
```

The model can also be plotted using the *plot_model()* method. Figure 1 shows the plotted model.

## Compiling and training a model

After a model is built, it is first compiled and then trained using the training data set prepared. The following is the sample code to compile the model:

```
new.compile(loss='binary_crossentropy',
       optimizer='adam',metrics=['accuracy'])

The model is thereafter trained using the fit_generator()
methods, where the number of epochs and steps per epochs are
specified:

new.fit_generator(training_image_generator,
               steps_per_epoch = no_train_image // 32,
                           epochs=5,
                           validation_data=val_datagen,
                    validation_steps= no_val_image // 32,
                           verbose=1)
```

## Model evaluation

After successful training, the model now needs to be evaluated using a test data set. The following is the code to evaluate the CNN model:

```
new.evaluate_generator(test_img_generator, no_test_image // 32)
```

It returns the values for two parameters — accuracy and loss. These parameters are used to evaluate the performance and efficiency of the model. END

### References


[1] *https://www.guru99.ccom/tensorflow-vs-keras.html#1*
[2] *https://medium.com/implodinggradients/tensorflow-or-keras-which-one-should-i-learn-5dd7fa3f9ca0s*
[3] *https://towardsdatascience.com/building-a-convolutional-neural-network-cnn-in-keras-329fbbadc5f5*
[4] *https://missinglink.ai/guides/convolutional-neural-networks/python-convolutional-neural-network-creating-cnn-keras-tensorflow-plain-python/*
[5] *https://medium.com/@ksusorokina/image-classification-with-convolutional-neural-networks-496815db12a8*



**By: Dr Sanskruti Patel and Dr Atul Patel**

**Dr Sanskruti Patel** is an associate professor in the Faculty of Computer Science and Applications, Charusat, Gujarat.

**Dr Atul Patel** is the dean and a professor in the Faculty of Computer Science and Applications, Charusat, Gujarat.